

Objetivo

Desplegar dos aplicaciones que representen los componentes frontend y backend de una solución. Crear Services ClusterIP para proporcionar acceso interno estable hacia Pods administrados por Deployments. Exponer el frontend mediante un Service NodePort y comprobar el acceso desde el equipo host. Habilitar el Ingress Controller de Minikube y crear reglas de enrutamiento por path hacia múltiples Services. Validar el flujo completo Pod → Service → Ingress y reconocer los puntos principales de verificación.

Requisitos previos

- Haber completado la Práctica 5 y la Práctica complementaria 5.1.
- Tener Docker Desktop, Minikube y kubectl disponibles.
- Utilizar Visual Studio Code con Git Bash como terminal integrada.
- Conservar el clúster Minikube de tres nodos utilizado en prácticas anteriores.
- Comprender Pods, Deployments, labels, selectors y puertos TCP.

Duración estimada

70 minutos

Introducción

En esta práctica construirás el flujo de exposición de una aplicación Kubernetes utilizando Services e Ingress. Desplegarás un frontend y un backend, crearás Services ClusterIP para comunicación interna, expondrás el frontend mediante NodePort y habilitarás el NGINX Ingress Controller de Minikube para enrutar las rutas / y /api hacia Services diferentes. Las prácticas complementarias posteriores profundizarán en diagnóstico de Services, endpoints y DNS interno.

Instrucciones

Preparación del directorio y namespace

En esta práctica crearás un namespace independiente y los manifiestos necesarios para construir la arquitectura de red.

Preparar el entorno

- **Paso 1.** Abre **Docker Desktop** y confirma que el motor está activo antes de utilizar Minikube y desplegar los recursos.
- **Paso 2.** Abre **Visual Studio Code**, selecciona **Git Bash** como terminal integrada y crea el directorio del laboratorio.

```
mkdir -p /c/LABS/kubernetes/lab6  
cd /c/LABS/kubernetes/lab6
```

 Copiar

- **Paso 3.** Confirma que Minikube continúa activo y que los tres nodos están disponibles antes de crear los workloads.

```
minikube status  
kubectl get nodes
```

 Copiar

Salida esperada aproximada:

```
minikube
```

```
type: Control Plane
```

```
host: Running
```

```
kubelet: Running
```

```
apiserver: Running
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	...	...
minikube-m02	Ready	<none>	...	...
minikube-m03	Ready	<none>	...	...

 Copiar

Los nombres exactos, versión y antigüedad pueden variar, pero los tres nodos deben estar en estado **Ready**.

- **Paso 4.** Crea el namespace **lab6** para aislar todos los Deployments, Services e Ingress utilizados durante esta práctica.

```
kubectl create namespace lab6
```

 Copiar

Salida esperada:

```
namespace/lab6 created
```

 Copiar

- **Paso 5.** Configura `lab6` como namespace predeterminado del contexto activo para reducir errores en los comandos posteriores.

```
kubectl config set-context --current --  
namespace=lab6
```

 Copiar**Salida esperada aproximada:**

```
Context "<nombre-del-contexto>" modified.
```

 Copiar

- **Paso 6.** Verifica que el contexto activo utiliza el namespace correcto para evitar crear recursos accidentalmente en `default`.

```
kubectl config view --minify | grep namespace
```

 Copiar**Salida esperada:**

```
namespace: lab6
```

 Copiar **Tarea 1. Desplegar frontend y backend**

En esta tarea crearás dos Deployments con respuestas HTML diferentes para poder identificar visualmente qué componente procesa cada petición.

Tarea 1.1. Crear el backend

- **Paso 7.** Crea el archivo `backend.yaml` para definir dos réplicas de NGINX que representarán una API interna.

```
touch backend.yaml
```

 Copiar

- **Paso 8.** Abre el archivo en VS Code y agrega el Deployment siguiente.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: lab6
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          command:
            - /bin/sh
            - -c
          args:
            - |
              echo "<html><body><h1>BACKEND
API</h1><p>Pod: $(hostname)</p></body></html>" >
/usr/share/nginx/html/index.html
              nginx -g 'daemon off;'
```

Tarea 1.2. Crear el frontend

- **Paso 9.** Crea el archivo `frontend.yaml` y define dos réplicas para representar la capa web de la aplicación.

```
touch frontend.yaml
```

- **Paso 10.** Agrega el manifiesto siguiente.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: lab6
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          command:
            - /bin/sh
            - -c
          args:
            - |
              echo "<html><body><h1>FRONTEND
WEB</h1><p>Pod: $(hostname)</p></body></html>" >
              /usr/share/nginx/html/index.html
              nginx -g 'daemon off;'
```

 Copiar

Tarea 1.3. Aplicar y validar

- **Paso 11.** Valida ambos manifiestos antes de crearlos para detectar problemas de sintaxis o estructura.

```
kubectl apply --dry-run=client -f backend.yaml
kubectl apply --dry-run=client -f frontend.yaml
```

 Copiar

Salida esperada:

```
deployment.apps/backend created (dry run)
deployment.apps/frontend created (dry run)
```

 Copiar

- **Paso 12.** Aplica los Deployments y espera hasta que sus réplicas se encuentren disponibles.

```
kubectl apply -f backend.yaml
kubectl apply -f frontend.yaml
```

 Copiar

Salida esperada:

```
deployment.apps/backend created
deployment.apps/frontend created
```

 Copiar

- **Paso 13.** Espera a que ambos Deployments completen su rollout y alcancen las dos réplicas disponibles antes de probar conectividad.

```
kubectl rollout status deployment/backend
kubectl rollout status deployment/frontend
```

 Copiar

Salida esperada:

```
deployment "backend" successfully rolled out
deployment "frontend" successfully rolled out
```

 Copiar

- **Paso 14.** Consulta los Pods con información extendida y observa sus IPs y nodos de ejecución.

```
kubectl get pods -o wide
```

 Copiar

Salida esperada aproximada:

NAME	READY	STATUS	IP	 Copiar
NODE				
backend-<hash>-<id>	1/1	Running		
10.244.x.x minikube-...				
backend-<hash>-<id>	1/1	Running		
10.244.x.x minikube-...				
frontend-<hash>-<id>	1/1	Running		
10.244.x.x minikube-...				
frontend-<hash>-<id>	1/1	Running		
10.244.x.x minikube-...				

Deben existir cuatro Pods **Running** y **Ready** : dos de **backend** y dos de **frontend** . La distribución entre nodos puede variar.

Tarea finalizada

Resultado esperado: Frontend y backend desplegados correctamente mediante Deployments con dos réplicas disponibles por componente.

Prompt de apoyo para explicar lo realizado

[➤ Explicar la Tarea 1 con IA](#)

Tarea 2. Crear Services ClusterIP

En esta tarea proporcionarás identidades de red estables para frontend y backend. Los Pods pueden cambiar de IP, mientras un Service conserva una dirección virtual y selecciona los Pods mediante labels.

Tarea 2.1. Crear los Services

- **Paso 15.** Crea el archivo `services-clusterip.yaml` para definir un Service interno para cada componente.

```
touch services-clusterip.yaml
```

 Copiar

- **Paso 16.** Agrega los dos Services siguientes, ambos de tipo **ClusterIP** .

```
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: lab6
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
  namespace: lab6
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
```

 Copiar

- **Paso 17.** Valida y aplica los dos Services, comprobando primero que el manifiesto sea aceptado por el cliente de Kubernetes.

```
kubectl apply --dry-run=client -f services-
clusterip.yaml
```

 Copiar

Salida esperada:

```
service/backend-svc created (dry run)
service/frontend-svc created (dry run)
```

 Copiar

- **Paso 18.** Aplica los Services y revisa las ClusterIP asignadas, que proporcionarán direcciones estables independientes de las IP de los Pods.

```
kubectl apply -f services-clusterip.yaml
kubectl get services -o wide
```

 Copiar

Salida esperada aproximada:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
PORT(S)			
backend-svc	ClusterIP	10.96.x.x	<none>
80/TCP			
frontend-svc	ClusterIP	10.96.y.y	<none>
80/TCP			

 Copiar

- **Paso 19.** Comprueba que cada Service descubrió dos backends antes de realizar solicitudes, validando la relación entre selectors y labels.

```
kubectl get endpoints backend-svc frontend-svc
```

 Copiar

Salida esperada aproximada:

NAME	ENDPOINTS
backend-svc	10.244.x.x:80,10.244.x.x:80
frontend-svc	10.244.x.x:80,10.244.x.x:80

 Copiar

Las IP exactas cambian entre ejecuciones; cada Service debe mostrar dos endpoints.

Tarea 2.2. Validar conectividad interna básica

- **Paso 20.** Obtén el nombre de uno de los Pods frontend para utilizarlo como origen de una prueba interna.

```
FRONTEND_POD=$(kubectl get pod \
  -l app=frontend \
  -o jsonpath='{.items[0].metadata.name}')

echo "$FRONTEND_POD"
```

 Copiar

Salida esperada aproximada:

```
frontend-<hash>-<id>
```

 Copiar

- **Paso 21.** Desde el Pod frontend, realiza una petición HTTP hacia `backend-svc` utilizando el nombre estable del Service.

```
kubectl exec "$FRONTEND_POD" -- \
  sh -c 'wget -qO- http://backend-svc'
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>BACKEND API</h1><p>Pod: backend-...
</p></body></html>
```

 Copiar

- **Paso 22.** Ejecuta varias solicitudes para comprobar que el Service puede dirigir tráfico hacia las réplicas disponibles del backend.

```
for i in 1 2 3 4; do
  kubectl exec "$FRONTEND_POD" -- sh -c 'wget -
qO- http://backend-svc' | grep -o 'Pod: [^<]*'
done
```

 Copiar

Salida esperada aproximada:

```
Pod: backend-<hash>-<id>
Pod: backend-<hash>-<id>
Pod: backend-<hash>-<id>
Pod: backend-<hash>-<id>
```

 Copiar

Pueden aparecer uno o ambos Pods backend durante las cuatro solicitudes.

NOTA: No es obligatorio observar una alternancia exacta entre Pods en cuatro solicitudes. Kubernetes distribuye conexiones entre endpoints disponibles, pero una secuencia corta no garantiza reparto perfectamente uniforme.

Tarea finalizada

Resultado esperado: Services ClusterIP configurados para frontend y backend, proporcionando acceso interno estable hacia los Pods seleccionados.

Tarea 3. Exponer el frontend mediante NodePort

En esta tarea crearás un segundo Service para frontend que pueda ser alcanzado desde fuera del clúster mediante un puerto asignado en los nodos.

Tarea 3.1. Crear el NodePort

- **Paso 23.** Crea el archivo `frontend-nodeport.yaml` y define un Service que exponga el frontend mediante el puerto `30080`.

```
touch frontend-nodeport.yaml
```

[Copiar](#)

- **Paso 24.** Agrega el manifiesto siguiente.

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-nodeport
  namespace: lab6
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

 Copiar

- **Paso 25.** Valida el manifiesto antes de crearlo para confirmar que `30080` pertenece al rango estándar de NodePort y que la estructura es correcta.

```
kubectl apply --dry-run=client -f frontend-
nodeport.yaml
```

 Copiar

Salida esperada:

```
service/frontend-nodeport created (dry run)
```

 Copiar

- **Paso 26.** Aplica el manifiesto y comprueba la relación entre `port: 80`, `targetPort: 80` y el puerto externo `30080`.

```
kubectl apply -f frontend-nodeport.yaml
kubectl get service frontend-nodeport
```

 Copiar

Salida esperada aproximada:

NAME	TYPE	CLUSTER-IP
EXTERNAL-IP	PORT(S)	
frontend-nodeport	NodePort	10.96.x.x
80:30080/TCP		<none>

 Copiar

- **Paso 27.** Comprueba que el NodePort selecciona las dos réplicas frontend antes de intentar acceder desde el equipo host.

```
kubectl get endpoints frontend-nodeport
```

 Copiar

Salida esperada aproximada:

NAME	ENDPOINTS
frontend-nodeport	10.244.x.x:80,10.244.x.x:80

 Copiar

Tarea 3.2. Obtener una URL accesible desde el host

- **Paso 28.** Abre una segunda terminal Git Bash y ejecuta `minikube service --url`; con el driver Docker puede crearse un túnel local que debe permanecer activo durante la prueba.

```
minikube service frontend-nodeport \  
  --namespace=lab6 \  
  --url
```

 Copiar

Salida esperada aproximada:

```
http://127.0.0.1:<puerto-local>
```

 Copiar

En algunos entornos puede devolverse una dirección basada en la IP de Minikube. Utiliza exactamente la URL mostrada por el comando.

IMPORTANTE: Si el comando muestra mensajes indicando que la terminal debe permanecer abierta por utilizar el driver Docker, no la cierras hasta terminar la prueba con `curl`.

- **Paso 29.** Copia la URL generada y, desde la terminal principal, realiza una solicitud HTTP para comprobar acceso externo hacia el frontend.

```
curl -s URL_DEVUELTA_POR_MINIKUBE
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>FRONTEND WEB</h1><p>Pod: frontend-...</p></body></html>
```

 Copiar

La respuesta debe contener:

```
FRONTEND WEB
```

 Copiar

- **Paso 30.** Regresa a la segunda terminal y presiona `Ctrl+C` si `minikube service --url` mantiene un túnel activo después de completar la validación.

Tarea finalizada

Resultado esperado: Frontend expuesto mediante NodePort y accesible desde el host utilizando la URL proporcionada por Minikube.

Tarea 4. Habilitar Ingress y configurar enrutamiento por path

En esta tarea utilizarás Ingress para consolidar la exposición HTTP de frontend y backend bajo un único punto de entrada.

Tarea 4.1. Habilitar el Ingress Controller

- **Paso 31.** Habilita el add-on `ingress` de Minikube para instalar el NGINX Ingress Controller utilizado por las reglas HTTP.

```
minikube addons enable ingress
```

 Copiar

Salida esperada aproximada:

```
The 'ingress' addon is enabled
```

 Copiar

El texto exacto puede variar entre versiones de Minikube.

- **Paso 32.** Espera hasta que el Deployment del controlador indique que está disponible antes de aplicar reglas de Ingress.

```
kubectl rollout status deployment/ingress-nginx-  
controller -n ingress-nginx --timeout=180s
```

 Copiar

Salida esperada:

```
deployment "ingress-nginx-controller" successfully  
rolled out
```

 Copiar

- **Paso 33.** Comprueba que el controlador y la clase `nginx` existen en el clúster.

```
kubectl get pods -n ingress-nginx  
kubectl get ingressclass
```

 Copiar

Salida esperada aproximada:

```
NAME                                READY  
STATUS  
ingress-nginx-controller-...      1/1  
Running  
  
NAME    CONTROLLER  
nginx   k8s.io/ingress-nginx
```

 Copiar

Tarea 4.2. Crear reglas para frontend y backend

- **Paso 34.** Crea el archivo `ingress.yaml`, que enviará `/` hacia frontend y `/api` hacia backend.

```
touch ingress.yaml
```

 Copiar

- **Paso 35.** Agrega el manifiesto siguiente utilizando `ingressClassName: nginx`.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: lab6
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect:
"false"
    nginx.ingress.kubernetes.io/rewrite-target:
/
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-svc
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-svc
                port:
                  number: 80
```

 Copiar

- **Paso 36.** Valida y aplica el recurso Ingress para registrar las reglas de enrutamiento en el controlador NGINX.

```
kubectl apply --dry-run=client -f ingress.yaml
```

 Copiar

Salida esperada:

```
ingress.networking.k8s.io/app-ingress created (dry run)
```

 Copiar

- **Paso 37.** Aplica el Ingress después de validar el archivo para registrar las reglas `/` y `/api` en el clúster.

```
kubectl apply -f ingress.yaml
```

 Copiar**Salida esperada:**

```
ingress.networking.k8s.io/app-ingress created
```

 Copiar

- **Paso 38.** Revisa el recurso y confirma que cada path apunta al Service esperado.

```
kubectl get ingress app-ingress  
kubectl describe ingress app-ingress
```

 Copiar**Salida esperada aproximada:**

```
Name:          app-ingress  
Ingress Class: nginx  
Rules:  
  Path  Backends  
  ----  -  
  /api  backend-svc:80  
  /     frontend-svc:80
```

 Copiar

La salida puede incluir dirección, annotations y Events adicionales.

Tarea 4.3. Probar Ingress desde el host

- **Paso 39.** Abre una segunda terminal Git Bash y crea un port-forward temporal hacia el Service del Ingress Controller.

```
kubect1 port-forward \  
-n ingress-nginx \  
service/ingress-nginx-controller \  
8080:80
```

 Copiar

Salida esperada aproximada:

```
Forwarding from 127.0.0.1:8080 -> 80  
Forwarding from [::1]:8080 -> 80
```

 Copiar

Puede aparecer una sola línea o ambas, según la configuración de red del host.

IMPORTANTE: Mantén esta segunda terminal abierta mientras realizas las pruebas siguientes. `port-forward` solo se utiliza como puente local hacia el controlador; las decisiones `/` y `/api` siguen siendo realizadas por Ingress.

- **Paso 40.** Desde la terminal principal, realiza una petición a `/` para comprobar que Ingress dirige la solicitud hacia `frontend-svc`.

```
curl -s http://127.0.0.1:8080/
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>FRONTEND WEB</h1><p>Pod:  
frontend-...</p></body></html>
```

 Copiar

La respuesta debe contener `FRONTEND WEB`.

- **Paso 41.** Realiza una segunda petición a `/api` para comprobar que la regla correspondiente dirige el tráfico hacia `backend-svc`.

```
curl -s http://127.0.0.1:8080/api
```

 Copiar

Salida esperada aproximada:

```
<html><body><h1>BACKEND API</h1><p>Pod: backend-...</p></body></html>
```

 Copiar

La respuesta debe contener `BACKEND API`.

- **Paso 42.** Regresa a la segunda terminal y presiona `Ctrl+C` para cerrar el port-forward después de completar las pruebas.

Concepto clave: Ingress no reemplaza los Services. El controlador recibe la petición HTTP, evalúa las reglas y reenvía el tráfico al Service configurado como backend.

Tarea finalizada

Resultado esperado: Ingress Controller habilitado y reglas configuradas para enrutar `/` hacia frontend y `/api` hacia backend.

✓ Tarea 5. Validar la arquitectura completa

En esta tarea comprobarás que los componentes creados durante la práctica permanecen disponibles y correctamente relacionados.

Tarea 5.1. Revisar recursos

- **Paso 43.** Ejecuta la consulta siguiente para obtener una vista consolidada de Deployments, Pods y Services del namespace.

```
kubectl get all -n lab6
```

 Copiar

Salida esperada:

La consulta debe mostrar dos Deployments, cuatro Pods `Running` y los Services `backend-svc`, `frontend-svc` y `frontend-nodeport`.

- **Paso 44.** Comprueba que el Ingress existe y utiliza la clase `nginx`.

```
kubectl get ingress app-ingress -n lab6 -o wide
```

 Copiar

Salida esperada aproximada:

NAME	CLASS	HOSTS	ADDRESS	PORTS
app-ingress	nginx	*	...	80

 Copiar

El campo `ADDRESS` puede tardar en poblarse o permanecer vacío en Minikube; la validación funcional se realizó mediante port-forward.

Tarea 5.2. Ejecutar la validación automática

- **Paso 45.** Copia y ejecuta el bloque siguiente para validar réplicas, tipos de Service, NodePort, endpoints e Ingress sin depender de valores de IP dinámicos.

```
echo "=== Verificacion final de la Practica 6
=== "

BACKEND_READY=$(kubectl get deployment backend \
  -n lab6 \
  -o jsonpath='{.status.readyReplicas}')

FRONTEND_READY=$(kubectl get deployment frontend \
  -n lab6 \
  -o jsonpath='{.status.readyReplicas}')

[ "$BACKEND_READY" = "2" ] \
  && echo "✅ backend disponible: 2/2" \
  || echo "❌ backend incompleto:
${BACKEND_READY:-0}/2"

[ "$FRONTEND_READY" = "2" ] \
  && echo "✅ frontend disponible: 2/2" \
  || echo "❌ frontend incompleto:
${FRONTEND_READY:-0}/2"

BACKEND_TYPE=$(kubectl get service backend-svc \
  -n lab6 \
  -o jsonpath='{.spec.type}')

FRONTEND_TYPE=$(kubectl get service frontend-svc \
  -n lab6 \
  -o jsonpath='{.spec.type}')

NODEPORT_TYPE=$(kubectl get service frontend-
nodeport \
  -n lab6 \
  -o jsonpath='{.spec.type}')

[ "$BACKEND_TYPE" = "ClusterIP" ] \
  && echo "✅ backend-svc es ClusterIP" \
  || echo "❌ backend-svc tiene tipo inesperado:
$BACKEND_TYPE"

[ "$FRONTEND_TYPE" = "ClusterIP" ] \
  && echo "✅ frontend-svc es ClusterIP" \
  || echo "❌ frontend-svc tiene tipo
```

 Copiar

```
inesperado: $FRONTEND_TYPE"

[ "$NODEPORT_TYPE" = "NodePort" ] \
  && echo "✅ frontend-nodeport es NodePort" \
  || echo "❌ frontend-nodeport tiene tipo
inesperado: $NODEPORT_TYPE"

NODEPORT=$(kubectl get service frontend-nodeport \
  -n lab6 \
  -o jsonpath='{.spec.ports[0].nodePort}')

[ "$NODEPORT" = "30080" ] \
  && echo "✅ NodePort configurado en 30080" \
  || echo "❌ NodePort inesperado: $NODEPORT"

BACKEND_ENDPOINTS=$(kubectl get endpoints
backend-svc \
  -n lab6 \
  -o jsonpath='{.subsets[0].addresses[*].ip}')

FRONTEND_ENDPOINTS=$(kubectl get endpoints
frontend-svc \
  -n lab6 \
  -o jsonpath='{.subsets[0].addresses[*].ip}')

[ -n "$BACKEND_ENDPOINTS" ] \
  && echo "✅ backend-svc tiene endpoints" \
  || echo "❌ backend-svc no tiene endpoints"

[ -n "$FRONTEND_ENDPOINTS" ] \
  && echo "✅ frontend-svc tiene endpoints" \
  || echo "❌ frontend-svc no tiene endpoints"

INGRESS_CLASS=$(kubectl get ingress app-ingress \
  -n lab6 \
  -o jsonpath='{.spec.ingressClassName}')

[ "$INGRESS_CLASS" = "nginx" ] \
  && echo "✅ Ingress utiliza la clase nginx" \
  || echo "❌ IngressClass inesperada:
$INGRESS_CLASS"

ROOT_BACKEND=$(kubectl get ingress app-ingress \
```

```

-n lab6 \
-o jsonpath='{.spec.rules[0].http.paths[?
(@.path=="/*").backend.service.name}'')

API_BACKEND=$(kubectl get ingress app-ingress \
-n lab6 \
-o jsonpath='{.spec.rules[0].http.paths[?
(@.path=="/*").backend.service.name}'')

[ "$ROOT_BACKEND" = "frontend-svc" ] \
&& echo "✅ / apunta a frontend-svc" \
|| echo "❌ / apunta a: $ROOT_BACKEND"

[ "$API_BACKEND" = "backend-svc" ] \
&& echo "✅ /api apunta a backend-svc" \
|| echo "❌ /api apunta a: $API_BACKEND"

echo ""
echo "=== Fin de verificacion ==="

```

Salida esperada:

```

=== Verificacion final de la Practica 6 ===
✅ backend disponible: 2/2
✅ frontend disponible: 2/2
✅ backend-svc es ClusterIP
✅ frontend-svc es ClusterIP
✅ frontend-nodeport es NodePort
✅ NodePort configurado en 30080
Warning: v1 Endpoints is deprecated in v1.33+; use
discovery.k8s.io/v1 EndpointSlice
Warning: v1 Endpoints is deprecated in v1.33+; use
discovery.k8s.io/v1 EndpointSlice
✅ backend-svc tiene endpoints
✅ frontend-svc tiene endpoints
✅ Ingress utiliza la clase nginx
✅ / apunta a frontend-svc
✅ /api apunta a backend-svc

=== Fin de verificacion ===

```

 Copiar

IMPORTANTE: Conserva el namespace `lab6`, los Deployments, Services y el addon de Ingress. Las prácticas complementarias 6.1 y 6.2 reutilizarán estos recursos para diagnóstico de Services, endpoints y DNS.

Tarea finalizada

Resultado esperado: Arquitectura de exposición validada correctamente, confirmando Deployments, Services, NodePort e Ingress operativos.

Resolución de problemas

Problema 1. backend-svc no responde desde frontend

Síntoma: La petición `wget http://backend-svc` falla o queda esperando.

Causa probable: El Service no existe, los Pods backend no están disponibles o existe una inconsistencia entre selector y labels.

Solución:

```
kubectl get service backend-svc -n lab6
kubectl get pods -n lab6 -l app=backend --show-labels
kubectl describe service backend-svc -n lab6
kubectl get endpoints backend-svc -n lab6
```

[Copiar](#)

El Service debe mostrar endpoints en el puerto 80. Si aparecen vacíos, compara el selector `app=backend` con los labels reales de los Pods.

La práctica complementaria 6.1 profundizará en el diagnóstico sistemático de endpoints.

Problema 2. minikube service --url no permite acceder al NodePort

Síntoma: La URL no se genera, el comando permanece activo o `curl` no obtiene respuesta.

Causa probable: El Service no está disponible, Minikube no está activo o el túnel local del driver Docker no pudo establecerse.

Solución:

```
minikube status
kubectl get service frontend-nodeport -n lab6
kubectl get pods -n lab6 -l app=frontend
```

 Copiar

Vuelve a ejecutar `minikube service frontend-nodeport --namespace=lab6 --url`, conserva abierta la terminal si Minikube crea un túnel y utiliza exactamente la URL generada.

Problema 3. El Ingress Controller no alcanza Running

Síntoma: El rollout del controlador termina por timeout o los Pods permanecen en `ContainerCreating` o `ImagePullBackOff`.

Causa probable: Las imágenes del addon todavía se están descargando, existe un problema de conectividad o Docker Desktop no dispone de recursos suficientes.

Solución:

```
kubectl get pods -n ingress-nginx
kubectl get events -n ingress-nginx \
  --sort-by='.lastTimestamp'
```

 Copiar

Corrige el problema indicado por los eventos y repite el rollout antes de probar el Ingress.

Problema 4. Ingress devuelve 404

Síntoma: `curl http://127.0.0.1:8080/` o `/api` devuelve una respuesta 404 del controlador.

Causa probable: El recurso Ingress no fue aplicado, la clase no coincide con el controlador o el path solicitado no corresponde con las reglas definidas.

Solución:

```
kubectl get ingress app-ingress -n lab6
kubectl describe ingress app-ingress -n lab6
kubectl get ingressclass
```

 Copiar

Confirma que `ingressClassName` sea `nginx` y que existan las rutas `/` y `/api`.

Problema 5. Ingress devuelve 503 Service Unavailable

Síntoma: El controlador responde, pero una ruta devuelve `503 Service Unavailable`.

Causa probable: El Service utilizado como backend no dispone de Pods disponibles o no tiene destinos asociados.

Solución:

```
kubectl get services -n lab6
kubectl get pods -n lab6 --show-labels
kubectl describe service backend-svc -n lab6
kubectl describe service frontend-svc -n lab6
```

 Copiar

No cambies los selectors hasta identificar qué relación entre Service y Pods está fallando.

Problema 6. El port-forward no puede utilizar el puerto 8080

Síntoma: `kubectl port-forward` devuelve un error indicando que el puerto local `8080` ya está ocupado.

Causa probable: Otra aplicación o un port-forward anterior continúa escuchando en ese puerto del host.

Solución:

Utiliza otro puerto local sin cambiar el puerto 80 del Service del controlador.

```
kubectl port-forward \  
-n ingress-nginx \  
service/ingress-nginx-controller \  
8081:80
```

 Copiar

Después realiza las pruebas con:

```
curl -s http://127.0.0.1:8081/  
curl -s http://127.0.0.1:8081/api
```

 Copiar

El cambio afecta únicamente al puerto local del host; las reglas del Ingress permanecen iguales.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar la práctica habrás desplegado una arquitectura frontend/backend y comprobado tres niveles de exposición: acceso interno mediante ClusterIP, acceso externo mediante NodePort y enrutamiento HTTP centralizado mediante Ingress.

Notas y consideraciones

- Esta práctica utiliza un namespace dedicado llamado lab6.
- En Windows y macOS con el driver Docker, utiliza `minikube service --url` para probar NodePort de forma confiable.
- Para validar Ingress desde el host se utilizará un port-forward temporal hacia el Ingress Controller, evitando depender de rutas especiales del host hacia la red interna de Minikube.
- La resolución DNS detallada y el diagnóstico profundo de endpoints se trabajarán en prácticas complementarias.
- En Windows y macOS con el driver Docker, `minikube service --url` puede mantener un túnel activo; conserva esa terminal abierta mientras pruebas la URL.
- Las pruebas HTTP internas se realizan desde los Pods mediante nombres de Service y las pruebas de Ingress desde el host utilizan `127.0.0.1` mediante port-forward.

URL de referencia

- [Services](#)
- [Ingress](#)
- [DNS for Services and Pods](#)

- [Minikube Addons](#)